Established – 1961 Subject: IOT

# SEVA SADAN'S

# R. K. TALREJA COLLEGE

# OF

# ARTS, SCIENCE & COMMERCE

# ULHASNAGAR – 421 003

## CERTIFICATE

**This is to certify that Mr./Ms. Dhanshree Salve of S.Y. Computer Science (SYCS) Roll No. 252034 has satisfactorily completed The Internet Of Thing Mini Project entitled Air Quality Monitoring System during the academic year 2025 – 2026, as a part of the practical requirement.**
**The project work is found to be satisfactory and is approved for submission.**

**PROF. INCHARGE** **HEAD OF DEPT**

**Sahil Shukla** **Diksha Jhamnani**

# INDEX


| Sr. No. | Chapters | Page No. |
|---|---|---|
| 1 | Introduction | |
| 2 | Requirement Specification | |
| 3 | System Design | |
| 4 | Implementation | |
| 5 | System Testing and Result | |
| 6 | Future Scope and Conclusion | |
| 7 | References | |
| 8 | Glossary | |

# INTRODUCTION

## Introduction : Air Quality Monitoring system

The IoT Based Air Quality Monitoring System is a simple and practical project designed to measure and monitor the quality of air in real time using IoT technology. The system detects harmful gases and air pollution levels and displays the measured values on a web dashboard or display unit.

The project is built using an ESP32/ESP8266 microcontroller as the main controller, which interfaces with an air quality sensor such as MQ135. The sensor detects gases like carbon dioxide ($CO_2$), ammonia ($NH_3$), smoke, and other harmful pollutants present in the air.

The system uses WiFi connectivity to upload air quality data to a cloud platform or display it on a web server, allowing remote monitoring from a smartphone or computer.

This project provides a practical understanding of how IoT technology can be used in environmental monitoring by combining sensors, microcontrollers, and cloud platforms for real-time air quality tracking.

## ● Motivation :

The motivation behind developing the IoT Based Air Quality Monitoring System is the increasing concern about air pollution and its harmful effects on human health.

Poor air quality can lead to respiratory problems, allergies, asthma, and other serious health issues. Continuous monitoring of air pollution levels helps in maintaining a healthy environment and taking preventive measures.

This project also provides students with practical exposure to IoT concepts, environmental sensing, wireless communication, and real-time data monitoring.

---

## ● Problem Definition :

The main problem addressed by this project is the lack of affordable and real-time air quality monitoring systems for homes, classrooms, and small environments.

Traditional air monitoring systems can be expensive and not easily accessible for educational or small-scale use.

This project aims to design a low-cost system that can:

• Detect air pollution levels using an MQ135 sensor
• Process and display the measured values
• Upload data to a cloud platform using WiFi
• Allow remote monitoring through a web interface

The system helps in understanding how environmental data can be monitored, analyzed, and stored efficiently using IoT technology.

---

## How it Works :

The IoT Based Air Quality Monitoring System works using an ESP32 or ESP8266 microcontroller to interface with an air quality sensor.

The MQ135 sensor detects the concentration of harmful gases in the surrounding air and produces an analog output signal proportional to pollution levels.

The microcontroller reads this analog value, processes the data, and calculates the air quality level. The measured values are:

• Displayed on a web dashboard or OLED display
• Updated in real time
• Uploaded to a cloud platform such as ThingSpeak

Using WiFi connectivity, the system enables remote monitoring of air quality from anywhere. This combination of sensor input, data processing, and cloud connectivity allows effective real-time environmental monitoring.

## Key Features :

Air Quality Measurement :

Uses MQ135 gas sensor to detect harmful gases and measure pollution levels.

Web Dashboard / Display Output :

Displays real-time air quality readings on a web interface or OLED display.

IoT Cloud Integration :

Uploads air quality data to a cloud platform such as ThingSpeak for remote monitoring and data visualization.

Real-Time Monitoring :

Provides continuous updates of air pollution levels without manual intervention.

User-Friendly Design :

Simple hardware setup and easy-to-understand interface suitable for students and beginners.

Educational Value :

Helps students understand IoT concepts, sensor interfacing, environmental monitoring, and wireless data transmission.

---

## ● Scope of the Project :

The IoT Based Air Quality Monitoring System has wide scope in environmental monitoring and smart city applications.

The system provides a cost-effective way to monitor air pollution levels in real time. It can be used in homes, schools, offices, and laboratories to maintain healthy air conditions.

In academic environments, the project helps students understand practical implementation of IoT systems involving sensors, microcontrollers, and cloud platforms.

The system can be further enhanced by:

• Adding temperature and humidity sensors
• Integrating mobile application support
• Implementing alert systems for high pollution levels
• Adding advanced data analytics and visualization

---

- ## Academic Institutions :

The system can be used as a practical learning tool in schools and colleges. Students gain hands-on experience with IoT devices, environmental sensing, and web-based monitoring systems.

It helps in understanding real-world implementation of smart environmental monitoring systems.

---

- ## Businesses and Environmental Services :

Industries, factories, and offices can use this system to monitor indoor air quality. The collected data can help in maintaining safe environmental conditions.

Gyms, classrooms, and hospitals can also use this system for maintaining healthy indoor air.

Its low-cost design makes it suitable for small-scale environmental monitoring applications.

---

- ## Data Security and Environmental Monitoring :

The system can be improved to securely store environmental data on cloud platforms. This ensures proper data management and safe access for analysis and future reference.

Secure cloud storage enables historical data tracking and pollution trend analysis.

---

- ## Online Learning Platforms :

Online learning platforms can use this project to teach IoT-based environmental monitoring concepts. Students can build and test the system at home, making learning interactive and practical.

---

# Objective :

The objective of the IoT Based Air Quality Monitoring System is to design a simple, reliable, and cost-effective system capable of monitoring air pollution levels using IoT technology.

The project aims to provide an easy-to-use environmental monitoring solution while helping students understand IoT concepts and sensor integration.

---

Key Goals Include :

Promote Learning in IoT and Environmental Monitoring :

Encourage students to understand IoT, sensors, and environmental safety through practical implementation.

Accurate Air Quality Detection :

Develop a system that accurately detects pollution levels using MQ135 sensor.

Real-Time Monitoring :

Enable real-time display of air quality data and remote monitoring through cloud platforms.

User-Friendly Design :

Create a simple and easy-to-operate system suitable for beginners.

Cloud Data Storage :

Store and visualize environmental data using IoT cloud platforms.

Support Future Enhancements :

Provide a foundation for adding advanced features such as alerts, mobile apps, additional sensors, and smart automation systems.

# REQUIREMENT SPECIFICATION

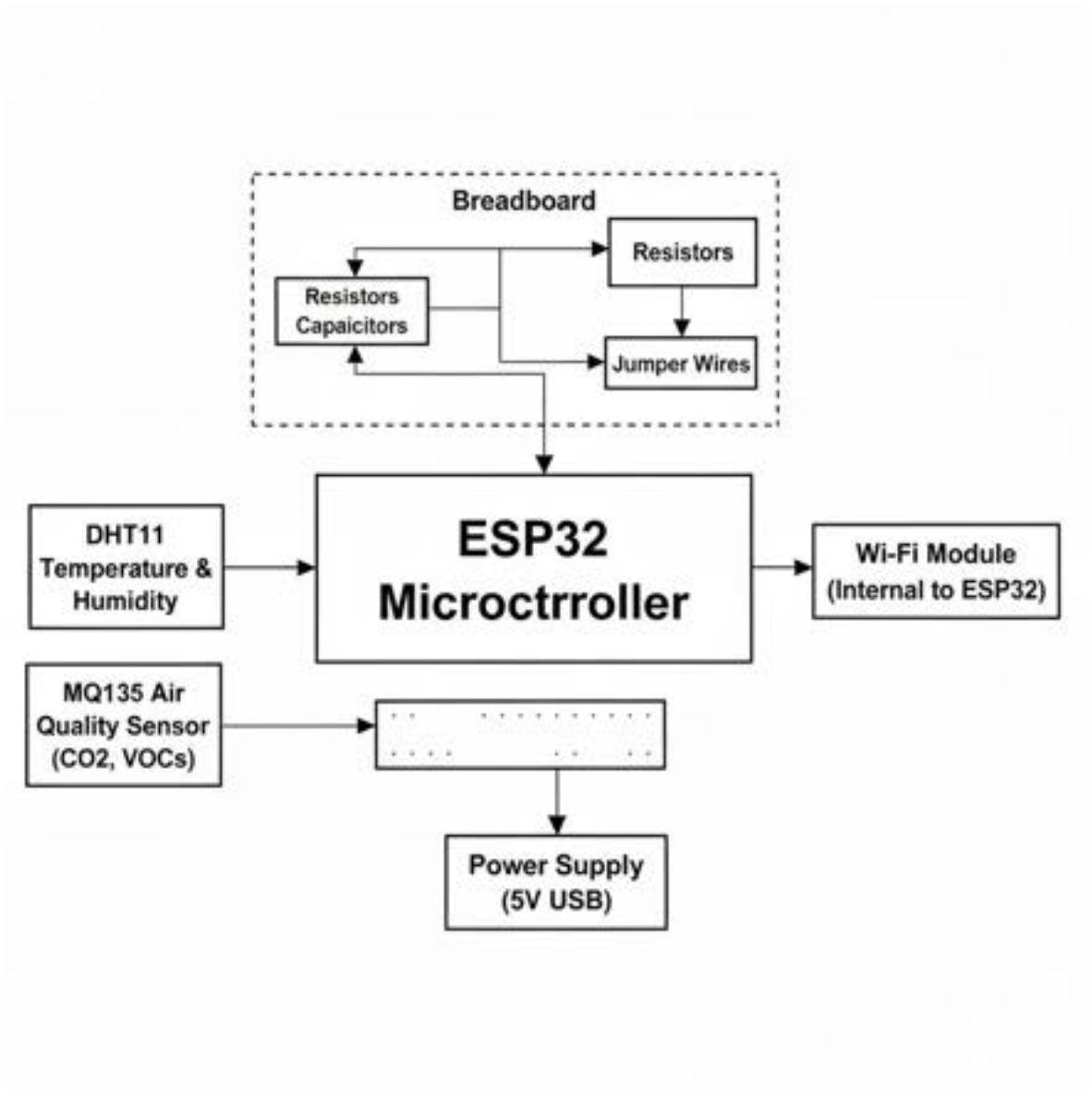
## Components Required

| Component | Specification |
|---|---|
| Microcontroller | ESP32 Development Board |
| Air Quality Sensor | MQ135 Gas Sensor for detecting air pollution |
| Temperature & Humidity Sensor | DHT11 Sensor |
| Breadboard | For mounting and testing components |
| Connecting Wires | Jumper wires for circuit connections |
| USB Data Cable | For programming and power supply |

| Component | Specification |
| --- | --- |
| Power Supply | USB power from laptop or 5V adapter |

# SYSTEM DESIGN

**Block Diagram** :




## 1. Input Stage (The Sensors)

These are the "senses" of your system. They collect raw environmental data and convert it into electrical signals.

- **DHT11 (Temperature & Humidity Sensor):** This sensor provides digital data. It measures the ambient temperature and the moisture level in the air.
- **MQ135 (Gas/Air Quality Sensor):** This is an analog sensor. It detects various harmful gases like $NH_3$, $NO_x$, alcohol, benzene, and smoke. It outputs a varying voltage based on the concentration of these gases.

## 2. Processing Stage (The Brain)

This is where the magic happens.

- **ESP32 Microcontroller:** This is the heart of your project. It performs several key tasks:
  - **Power Distribution:** It takes power from the USB (5V) and often regulates it for the sensors.
  - **Data Conversion:** It uses its internal **ADC (Analog-to-Digital Converter)** to understand the MQ135's voltage and reads the digital stream from the DHT11.
  - **Logic:** It compares the readings against your "Normal" vs. "Unhealthy" thresholds.

## 3. Support Stage (The Breadboard)

- **Breadboard & Jumper Wires:** Think of this as the "skeleton" and "nervous system." It provides the physical platform to connect everything without soldering.
- **Resistors:** These are used to "pull up" data lines (especially for the DHT11) to ensure the signal stays clean and readable for the ESP32.

## 4. Connectivity Stage

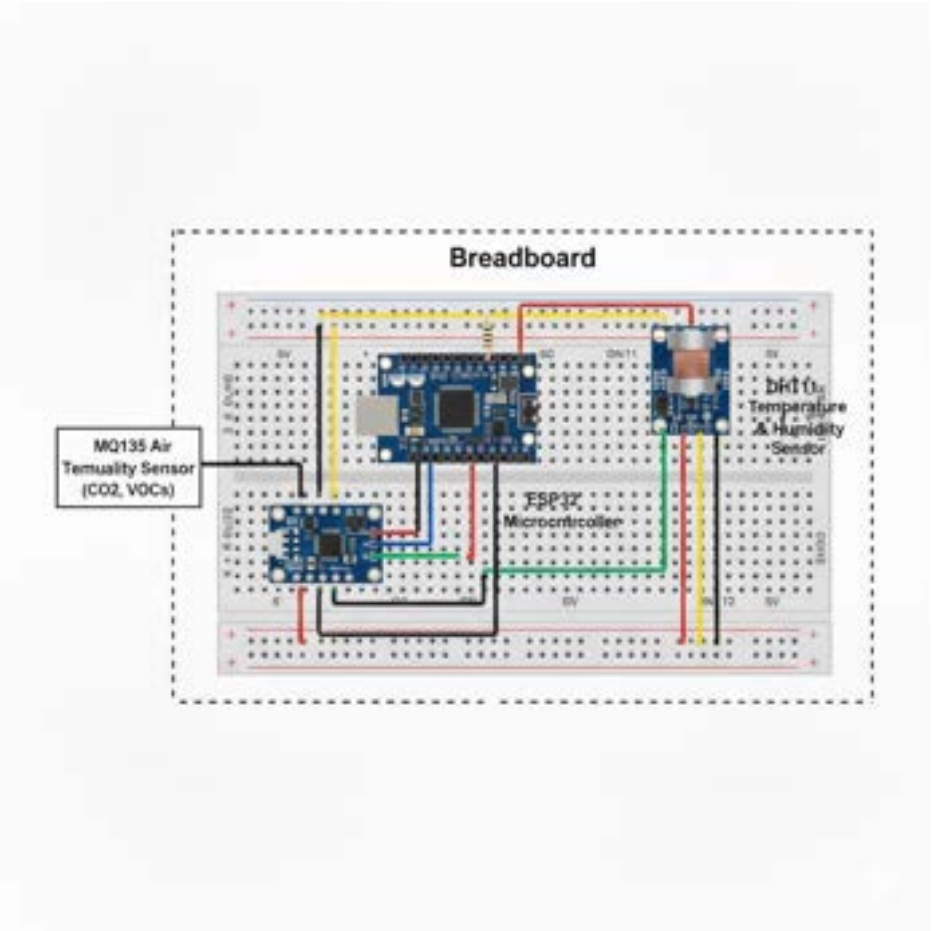
- **Internal Wi-Fi:** Since you're using an ESP32, the "Output" isn't just local. The block diagram includes the internal Wi-Fi module, which allows the device to communicate with other devices or a local network, even if you aren't sending data to a major cloud provider.

---

### How the Data Flows:

1. **Sensors** detect air changes.
2. **Breadboard** routes these signals to the **ESP32**.
3. **ESP32** processes the math (e.g., converting MQ135 voltage to PPM).
4. **System** stays ready to trigger an alert if the air quality drops.

**Circuit Diagram :**




## Project Description

The block diagram illustrates the interaction between the key components of the **IoT-Based Air Quality Monitoring System**. The **ESP32 microcontroller** acts as the central hub, receiving digital signals from the **DHT11** (Temperature & Humidity) and analog signals from the **MQ135** (Air Quality/Gas) sensors. The ESP32 processes these inputs to calculate environmental levels. While local logic determines if the air is "Normal" or "Unhealthy," the integrated Wi-Fi module allows the system to communicate data wirelessly. Power is supplied via a 5V USB connection to the ESP32, which then distributes power to the sensors through the breadboard rails.

---

## Key Steps of the Circuit Diagram

- **ESP32 Setup:** The ESP32 NodeMCU is placed across the center channel of the breadboard. It serves as the master controller for reading sensor data and managing timing.
- **MQ135 Sensor (Gas/Smoke):** The MQ135 VCC is connected to the 5V rail and GND to the ground rail. Its **Analog Output (A0)** pin is connected to an ADC pin on the ESP32 (e.g., GPIO 34) to measure gas concentrations.
- **DHT11 Sensor (Temp/Humidity):** Connected to the power rail (3.3V or 5V). Its **Data pin** is connected to a digital GPIO (e.g., GPIO 4). A pull-up resistor (typically $4.7k\Omega$ to $10k\Omega$) is used between VCC and the Data pin to ensure stable digital communication.
- **Breadboard Power Rails:** The ESP32 **VIN** (5V) and **GND** pins are bridged to the breadboard's side rails to provide a central power source for both sensors.
- **WiFi Connectivity:** The ESP32 utilizes its internal antenna to connect to a local 2.4GHz network, enabling the system to be monitored remotely or synced with local time for data logging.

---

## Benefits of the Circuit Diagram

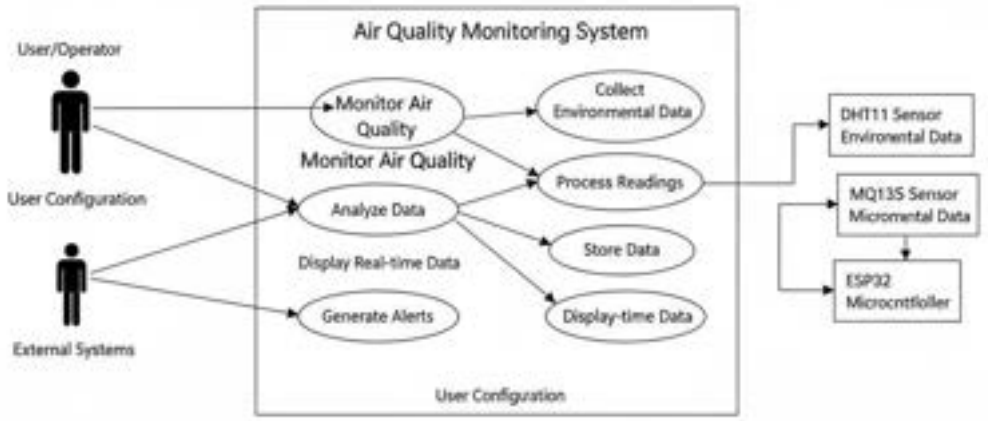
- **Clear Visualization:** Shows the exact wiring path from the sensors to the specific ESP32 GPIO pins.
- **Error Detection:** Helps identify "Short Circuits" (like accidentally connecting 5V to GND) before plugging in the USB.
- **Documentation:** Acts as a blueprint for your project completion by February 20, 2026.
- **Ease of Assembly:** Simplifies the process of placing jumper wires in the correct breadboard holes.

---

## Circuit Model Testing Methods

1. **Continuity Testing:** Before powering on, use a multimeter to ensure that ground (GND) is shared across all components and that there are no bridges between power and ground.
2. **Serial Monitor Debugging:** Power the ESP32 via USB and check the Arduino Serial Monitor. Verify that the DHT11 isn't returning "Nan" (Not a Number) and that the MQ135 is giving variable voltage readings.
3. **Functional Testing:** Expose the MQ135 to a known source (like a lighter's gas or smoke) and breathe on the DHT11 to see if the humidity and gas values increase in real-time.
4. **Stability Testing:** Run the system for several hours to ensure the ESP32 doesn't crash and that the sensors provide stable readings after the MQ135's initial "pre-heat" phase.

## Use Case Diagram :




## Description

- The Use Case Diagram for the **IoT-Based Air Quality Monitoring System** illustrates the interactions between the user and the system's functionalities. It provides a visual representation of the key actions performed by the system and how the user interacts with environmental data.
- The main actors identified in the system are the **User/Operator**, the **DHT11 & MQ135 Sensors**, and the **ESP32 Microcontroller**. Below is a detailed explanation of the use cases included in the diagram.

---

## Detailed Use Cases

- **Collect Environmental Data**
  - **Actor:** DHT11 Sensor, MQ135 Sensor
  - **Description:** This use case involves the sensors continuously scanning the environment for temperature, humidity, and harmful gas concentrations (CO2, VOCs, Smoke).
- **Process Readings**
  - **Actor:** ESP32 Microcontroller
  - **Description:** The system receives raw digital and analog signals. It processes these values to convert voltages into PPM (parts per million) for air quality and Celsius/Percentage for the DHT11.
- **Analyze Data & Determine Air Status**
  - **Actor:** ESP32 Microcontroller

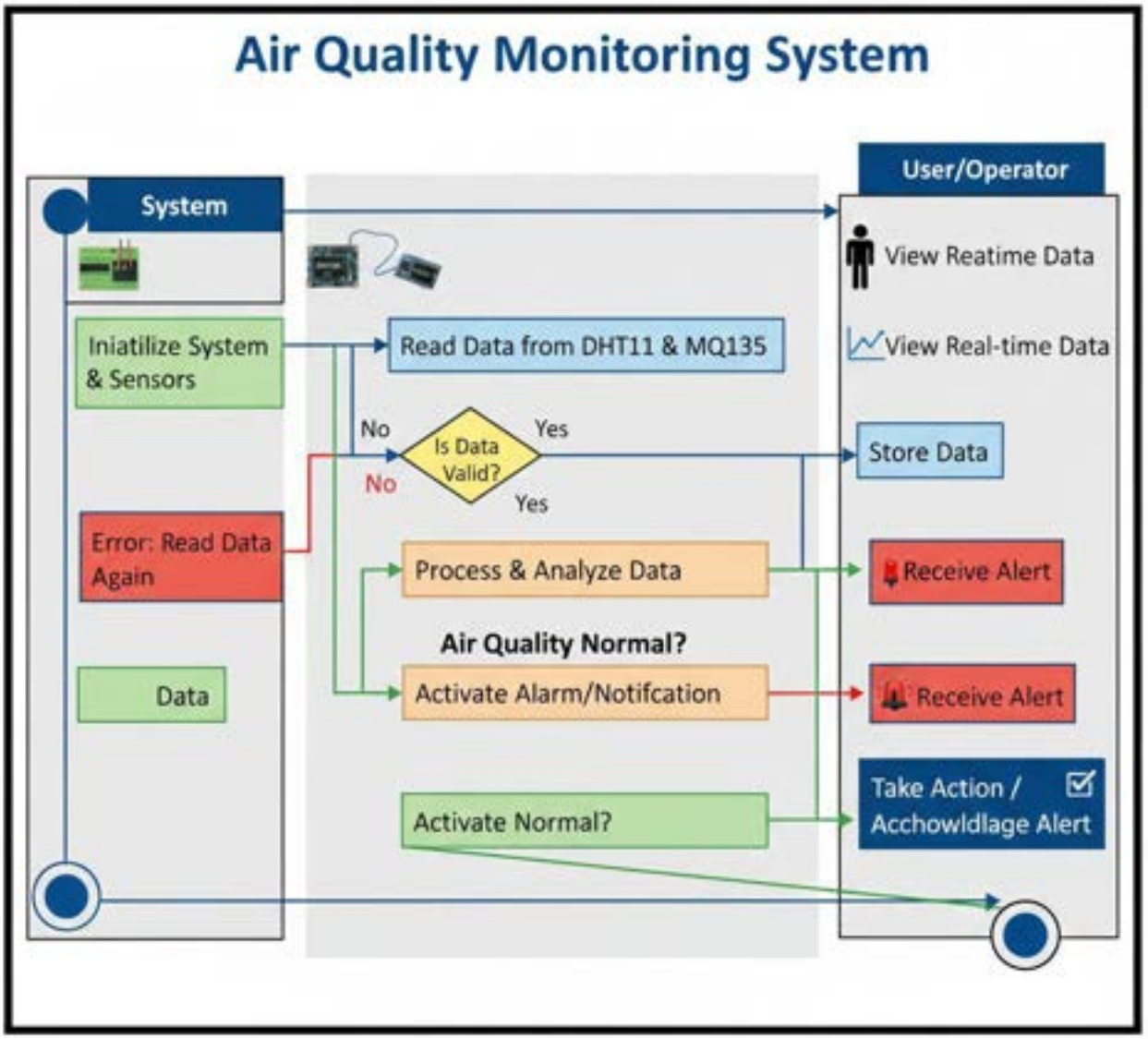
    - **Description:** The system compares the current readings against pre-defined safety thresholds. It determines if the air quality status is "Normal," "Moderate," or "Unhealthy."
- **Monitor Air Quality (Real-time View)**
    - **Actor:** User/Operator
    - **Description:** This allows the user to check current environmental conditions. Since we removed the OLED, this is typically done via the Serial Monitor or a local web server hosted on the ESP32.
- **Generate Alerts**
    - **Actor:** Air Quality Monitoring System
    - **Description:** If gas levels exceed the safety threshold, the system triggers an alert logic. This could be a visual signal (like an LED) or a digital notification to the user.
- **Store/Log Data**
    - **Actor:** ESP32 Microcontroller
    - **Description:** The system may temporarily store high/low values in the ESP32's memory to track fluctuations in air quality over a period of time.

---

## Purpose of Use Case Diagrams

- **Visualizing System Interactions:** Provides a clear visual representation of how the ESP32 manages data flow from the physical sensors to the user.
- **Defining System Requirements:** Helps identify and define functional requirements, ensuring that both gas detection and climate monitoring features are fully integrated.
- **Facilitating Communication:** Acts as an effective tool for project presentations by showing exactly what the "Air Monitoring System" is capable of doing.
- **Guiding System Design:** Highlights important functionalities, such as error handling (when a sensor is disconnected) and threshold logic.
- **Documenting Functional Requirements:** Serves as a permanent record of the system's behavior, which is essential for your final project submission on **February 20, 2026**.
- **Simplifying Complex IoT Logic:** Breaks down the relationship between hardware (sensors) and software (processing logic) into manageable, understandable interactions.

## Activity Diagram :




## Description

Activity diagrams are a type of UML (Unified Modeling Language) diagram that represent the flow of activities within a system. They are useful for showing how different actions are performed step by step. In the IoT-Based Air Quality Monitoring System, the activity diagram explains the complete workflow starting from sensor initialization to processing atmospheric data and triggering alerts. It helps in understanding how the system behaves during real-time operation, especially how it handles decision-making when gas levels become unsafe.

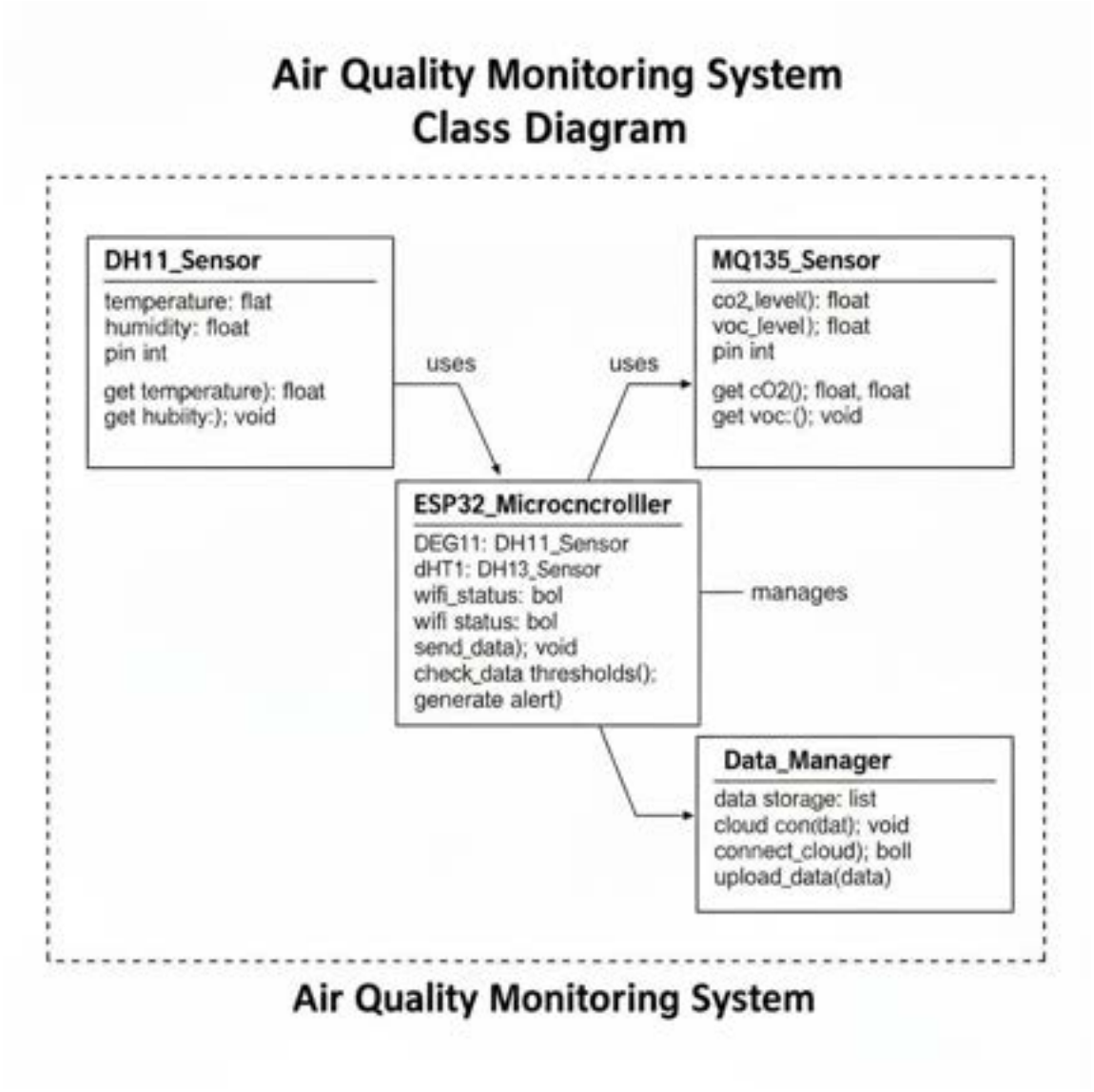
---

## Purpose of Activity Diagrams

- Visualize Workflow: Activity diagrams provide a clear visual representation of the workflow of the air quality system. They show the sequence of activities such as system start, sensor calibration, gas/temperature detection, threshold comparison, and alert generation, making the process easy to understand.

- **Modeling Processes:** These diagrams help in modeling the detailed processes involved in environmental monitoring. They describe how the ESP32 reads analog data from the MQ135 and digital data from the DHT11, processes these into readable units, and generates a status output, which is crucial during system development.
- **Identify Decision Logic:** Activity diagrams are excellent for representing decision points, such as checking if the **Gas PPM > Threshold**. If true, the system moves to the "Generate Alert" activity; if false, it continues normal monitoring. This helps in understanding the conditional operations within the system.
- **Clarifying Responsibilities:** The diagram clearly shows which component performs which task. For example, the **MQ135** detects gas concentration, the **ESP32** performs the data analysis and logic, and the **Alert System** provides the notification. This clarity helps in proper system implementation and troubleshooting.
- **Supporting Communication:** Activity diagrams act as an effective communication tool for your project presentation. They present the system workflow in a visual format that is easy to explain to instructors, even those who might not be familiar with the specific code logic of the ESP32.
- **Facilitating Documentation:** These diagrams serve as a vital part of your project documentation, finalized for your **February 20, 2026** deadline. They provide a reference for future improvements, such as adding a ventilation fan or a buzzer.
- **Enhancing System Design:** During the design phase, the activity diagram helps identify bottlenecks, such as the "pre-heat" time required for the MQ135 sensor, allowing you to build appropriate delays into your code before the actual monitoring begins.
- **Testing and Validation:** Activity diagrams assist in testing and validation by clearly showing expected system behavior. You can create test cases (e.g., "What happens when Smoke is detected?") based on the diagram to ensure the system responds reliably.

## Class Diagram :




## Description

Class diagrams are a fundamental part of object-oriented design, providing a visual representation of a system's classes and their relationships. They serve as a blueprint for the structure of a software application, illustrating how different classes interact with each other. In the context of the **IoT-Based Air Quality Monitoring System**, class diagrams play an important role in organizing and defining the system components such as the ESP32 microcontroller, environmental sensors, and data management logic. The diagram helps in understanding how data is abstracted from physical hardware into digital variables, ensuring clarity during the coding and implementation phase.

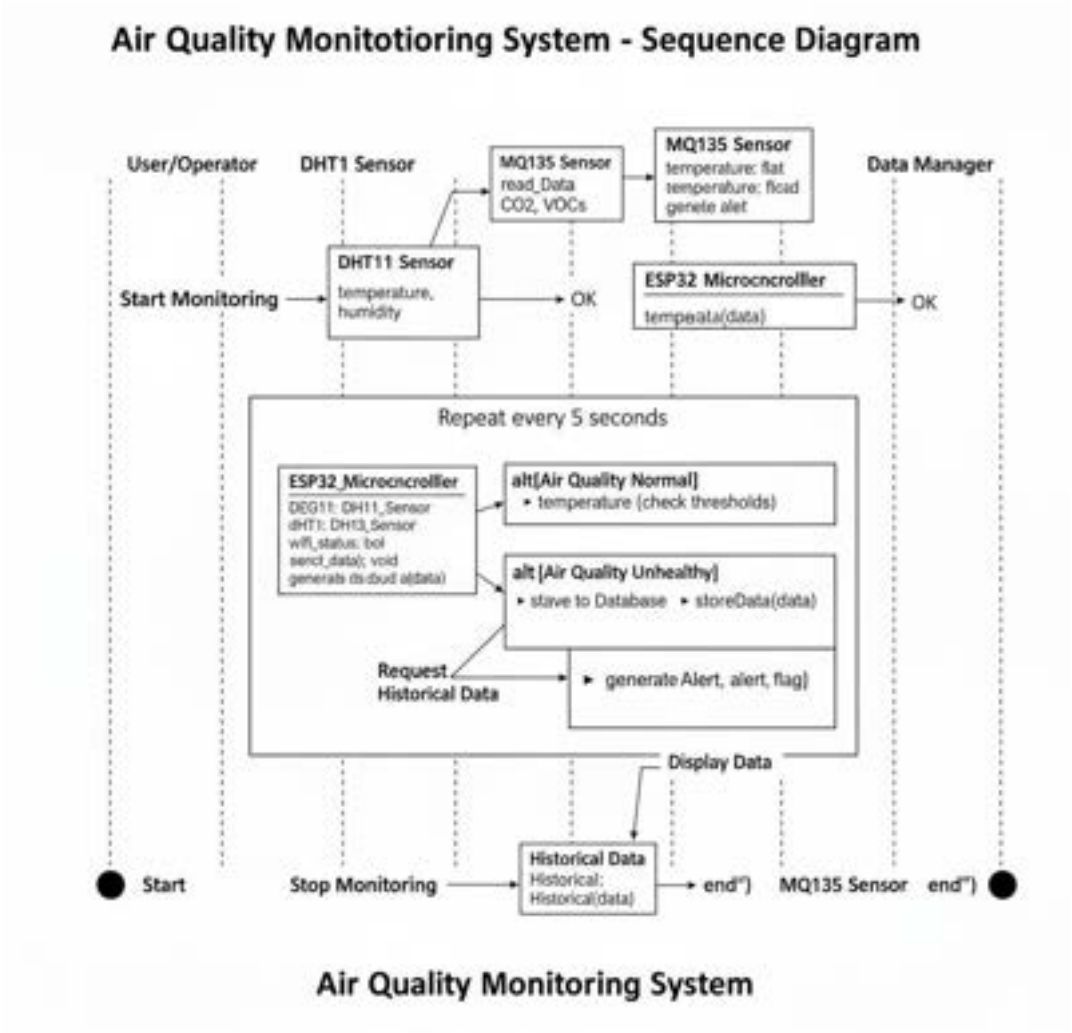
---

## Purpose of Class Diagrams

- **Modeling System Structure:** The class diagram visualizes the main components of the air monitoring system, including the **ESP32_Microcontroller**, **DHT11_Sensor**, **MQ135_Sensor**, and **Data_Manager**. It provides a clear structural overview of how hardware attributes (like pin numbers) and methods (like readSensor()) are organized.
- **Understanding Relationships:** It shows how different classes interact—for instance, how the ESP32 class **uses** the Sensor classes to fetch data and **manages** the Data_Manager to handle thresholds. This helps in understanding dependencies and communication protocols (Analog vs. Digital) within the system.

- **Design Blueprint:** The class diagram acts as a blueprint for implementing the system logic in C++/Arduino code. It defines exactly what data types (e.g., `float` for temperature, `int` for gas PPM) and functions (e.g., `checkThresholds()`) need to be written to make the system functional.
- **Documentation:** It serves as an important reference for future maintenance, such as adding a new sensor or switching to a different cloud platform. Developers can easily understand the system architecture and extend functionality without rewriting the core logic, which is vital for your project documentation due on **February 20, 2026**.
- **Defining Attributes and Operations:** By listing specific variables (like `humidity` and `gas_level`) and operations (like `generateAlert()`), the class diagram ensures that the software logic matches the physical capabilities of your sensors.

## Sequence Diagram :




## Description

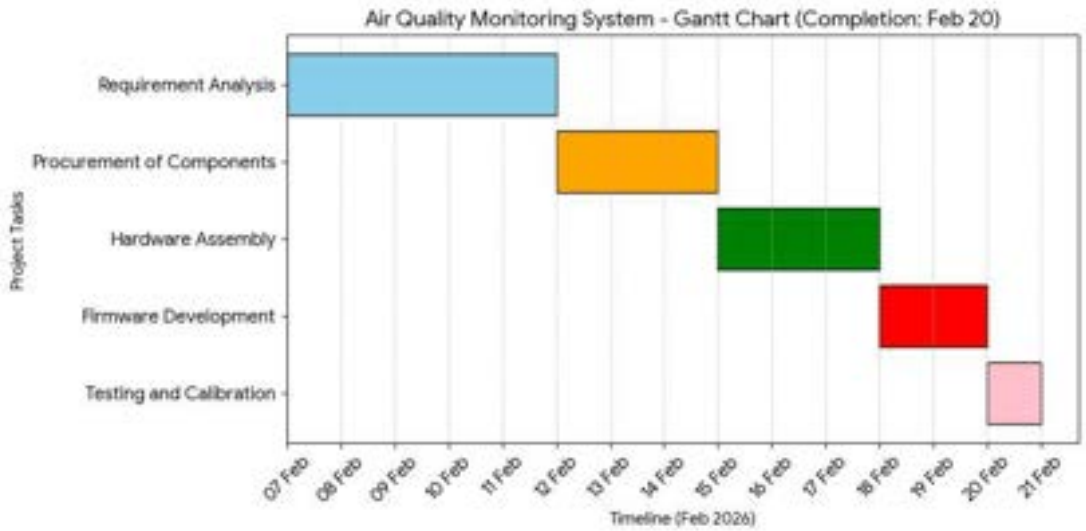
A Sequence Diagram is a type of interaction diagram in Unified Modeling Language (UML) that illustrates how different components of a system interact with each other in a specific sequence over time. It focuses on the order in which messages or data are exchanged between objects to complete a particular process. In the **IoT-Based Air Quality Monitoring System**, the sequence diagram helps in understanding how environmental data flows from the DHT11 and MQ135 sensors to the ESP32 microcontroller, and how that information is then used to update local displays or trigger alerts, showing the dynamic, real-time behavior of the system.

## Purpose of Sequence Diagrams

- Visual Representation of Interactions: Sequence diagrams provide a clear visual representation of interactions between system components such as the DHT11, MQ135, ESP32 Microcontroller, and Data Manager in a time-based sequence. It maps out the "life-line" of each component during a single monitoring cycle.
- Clarification of Message Flow: They help clarify how air quality data is sensed, processed, and evaluated. This makes it easier to understand the flow of control—starting from the initial sensor request to the final decision logic that determines if an alert is necessary.
- Facilitation of Communication: The diagram acts as a communication tool for your project, helping you explain the system workflow and data exchange clearly to instructors or peers. It bridges the gap between the static circuit design and the active code execution.
- Guidance for Development: It serves as a guide for implementing interactions between hardware and software. You can follow the sequence to program the exact timing of when the ESP32 should poll the sensors and how long it should wait before uploading or displaying the results to prevent data collisions.
- Documentation: Sequence diagrams provide clear documentation of system operation, making future maintenance, debugging, and enhancements easier. This ensures your project documentation is professional and thorough for your February 20, 2026 milestone.
- Identification of Issues: By visualizing the interaction flow, sequence diagrams help identify potential delays (like the MQ135 sensor warm-up time) or incorrect data transfer sequences, improving the overall reliability of your Air Quality Monitoring System.

.

## Gantt Chart :




## Description

A Gantt Chart is a type of bar chart that represents a project schedule. It visually outlines the tasks involved in a project along a timeline, showing when each task starts, how long it will take, and when it should be completed. Gantt charts are widely used in project management to plan, coordinate, and track specific tasks or activities, providing a clear overview of the project's progress and deadlines.

The Gantt chart below provides a visual representation of the timeline for the IoT-Based Air Quality Monitoring System project. The project tasks are scheduled between 7 February 2026 and 20 February

2026. The chart breaks the project into key phases such as requirement analysis, procurement, hardware assembly, firmware development, and testing/calibration, clearly displaying each phase's start and end dates.

This representation helps in understanding how the project was completed within a short time frame while ensuring proper planning and timely execution of all activities to meet the final submission deadline of February 20, 2026.

---

## Advantages of Using Gantt Charts

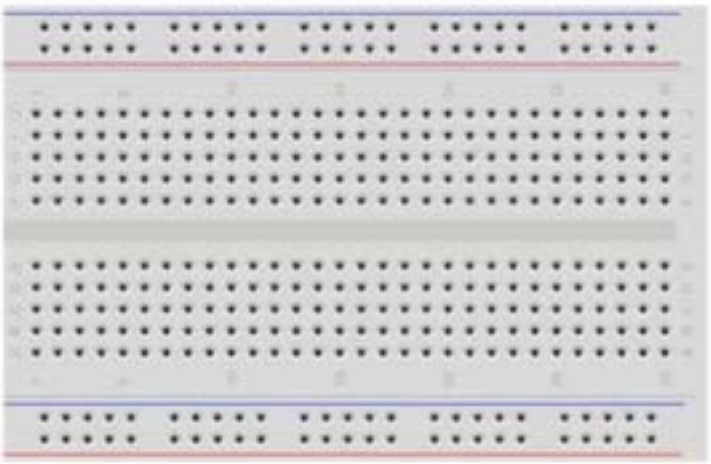
- Clear Visualization: Gantt charts provide a clear and simple visual representation of the project timeline, making it easy to understand the sequence of tasks—from researching the MQ135 sensor to final hardware calibration.
- Effective Planning: They help in organizing tasks efficiently by assigning specific time durations to each activity, ensuring that hardware assembly is completed before firmware development begins.
- Tracking Progress: Gantt charts allow continuous monitoring of project progress, helping identify which stages (like the DHT11 integration) are completed and which tasks (like final testing) are upcoming.
- Collaboration: They improve coordination by clearly defining timelines for each phase, ensuring that all components of the "iot" system are integrated systematically.
- Deadline Management: By setting a firm end date of February 20, the Gantt chart ensures that no single task (like procurement) takes too much time, keeping the entire project on track for a successful launch.

# SYSTEM IMPLEMENTATION

# Step-by-Step Assembly:

**Step 1 :**

Firstly, identify these components.



**Step 2 :**

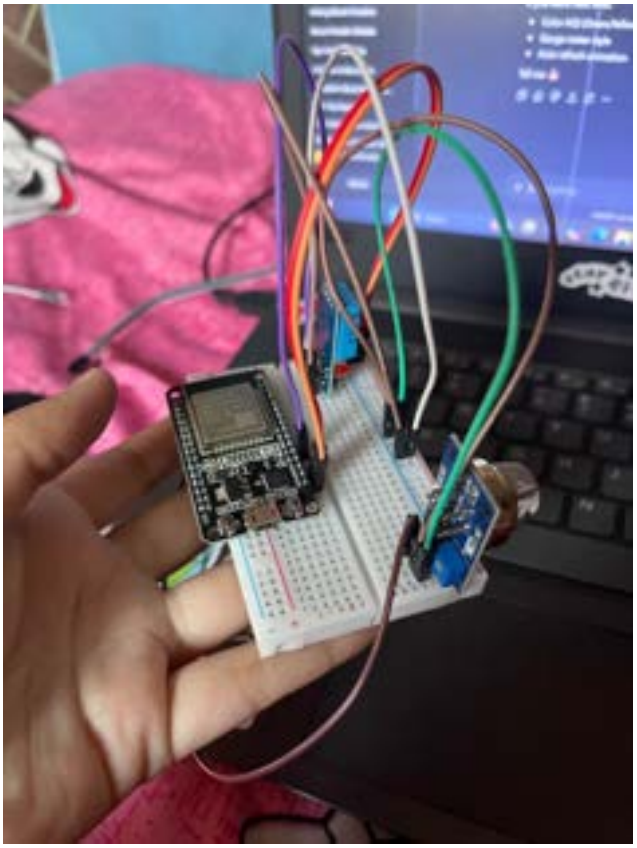
Place all components on Breadboard :

**Step 3 :**

Connect Pulse sensor jumper wires on following pins.

**Step 4 :**

Then, connect the jumper wires on following pins



**Step 5 :**

So, let's create the program for this project. This program includes all three functions. We can run these separately. It is as follows.

```
#include <WiFi.h>
#include <DHT.h>

#define DHTPIN 4
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);
const int mq135Pin = 34;

const char* ssid = "Unnati";
const char* password = "MICKEY0808";

WiFiServer server(80);

void setup() {
  Serial.begin(115200);
  dht.begin();

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) delay(500);
```

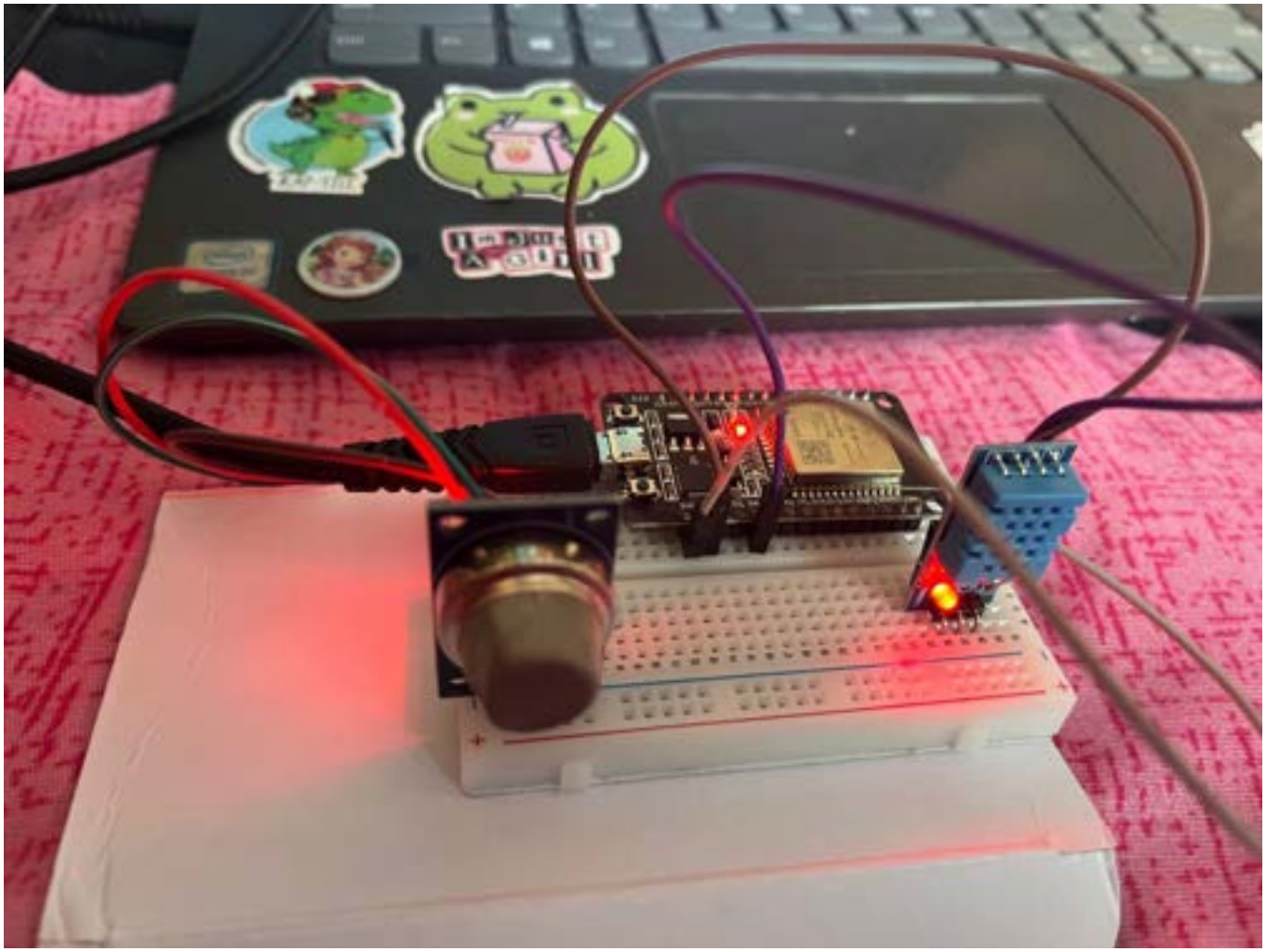
```
  Serial.println(WiFi.localIP());
  server.begin();
}

void loop() {
  WiFiClient client = server.available();

  if (client) {

    float h = dht.readHumidity();
    float t = dht.readTemperature();
    int air = analogRead(mq135Pin);

    client.println("HTTP/1.1 200 OK");
    client.println("Content-type:text/html; charset=UTF-8");
    client.println();

    client.println("<!DOCTYPE html><html><head>");
    client.println("<meta name='viewport' content='width=device-width, initial-
scale=1'>");

    client.println("<style>");
    client.println("@import
url('https://fonts.googleapis.com/css2?family=Poppins:wght@300;600&display=swap');");

    client.println("body{font-family:Poppins;background:#020617;color:white;text-
align:center;margin:0;}");

    client.println(".title{font-size:32px;font-weight:600;margin-top:20px;}");

    client.println(".card{background:#0f172a;border:2px solid #38bdf8;border-
radius:25px;padding:25px;margin:20px auto;width:340px;box-shadow:0 0 25px
rgba(56,189,248,0.6);}");

    client.println(".value{font-size:40px;font-weight:600;margin-
top:10px;color:#38bdf8;}");

    client.println(".label{font-size:20px;color:#cbd5e1;}");

    client.println(".gauge{width:260px;height:130px;background:#1e293b;border-
radius:260px 260px 0 0;overflow:hidden;margin:20px auto;position:relative;border:2px
solid #38bdf8;}");

    client.println(".needle{width:4px;height:125px;background:#ef4444;position:absolute
;bottom:0;left:50%;transform-origin:bottom center;}");

    client.println("button{padding:14px 28px;font-size:18px;border:none;border-
radius:15px;background:#38bdf8;color:black;margin:25px;font-weight:600;box-shadow:0 0
15px #38bdf8;}");

    client.println("</style>");

    client.println("<script>");
    client.println("setInterval(()=>location.reload(),3000);");
    client.println("</script>");
```

```
    client.println("</head><body>");

    client.println("<div class='title'>AIR QUALITY MONITOR</div>");

    client.println("<div class='card'>");
    client.println("<div class='label'>Temperature</div>");
    client.print("<div class='value'>");
    client.print(t);
    client.println(" C</div></div>");

    client.println("<div class='card'>");
    client.println("<div class='label'>Humidity</div>");
    client.print("<div class='value'>");
    client.print(h);
    client.println(" %</div></div>");

    client.println("<div class='card'>");
    client.println("<div class='label'>Air Quality</div>");

    client.println("<div class='gauge'>");
    client.print("<div class='needle' style='transform: rotate(");
    client.print(air / 12);
    client.println("deg);'></div>");
    client.println("</div>");

    client.print("<div class='value'>");
    client.print(air);
    client.println("</div></div>");

    client.println("<button onclick='location.reload()'>Refresh Data</button>");

    client.println("</body></html>");
    client.println();

    client.stop();
  }
}
```

# SYSTEM TESTING AND RESULT

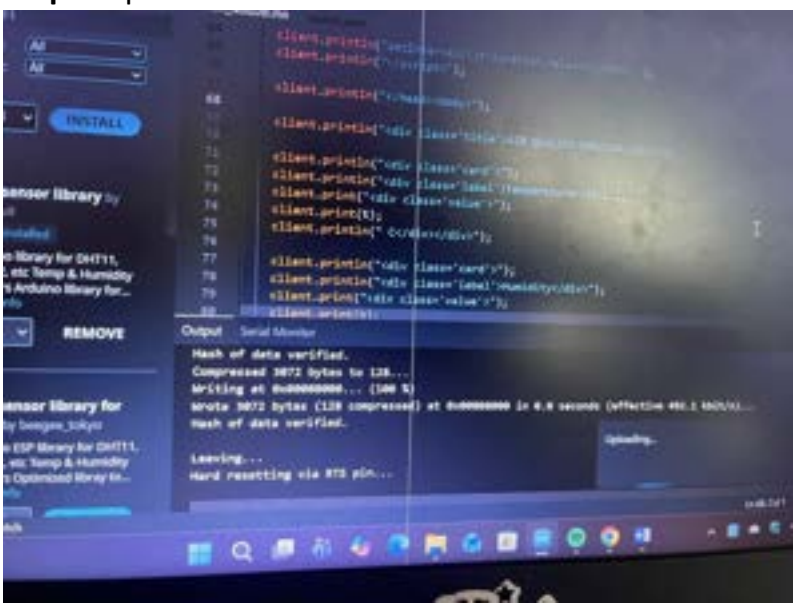
**Step 6 :**

Connect Datacable from Esp32 to laptop for code uploading.



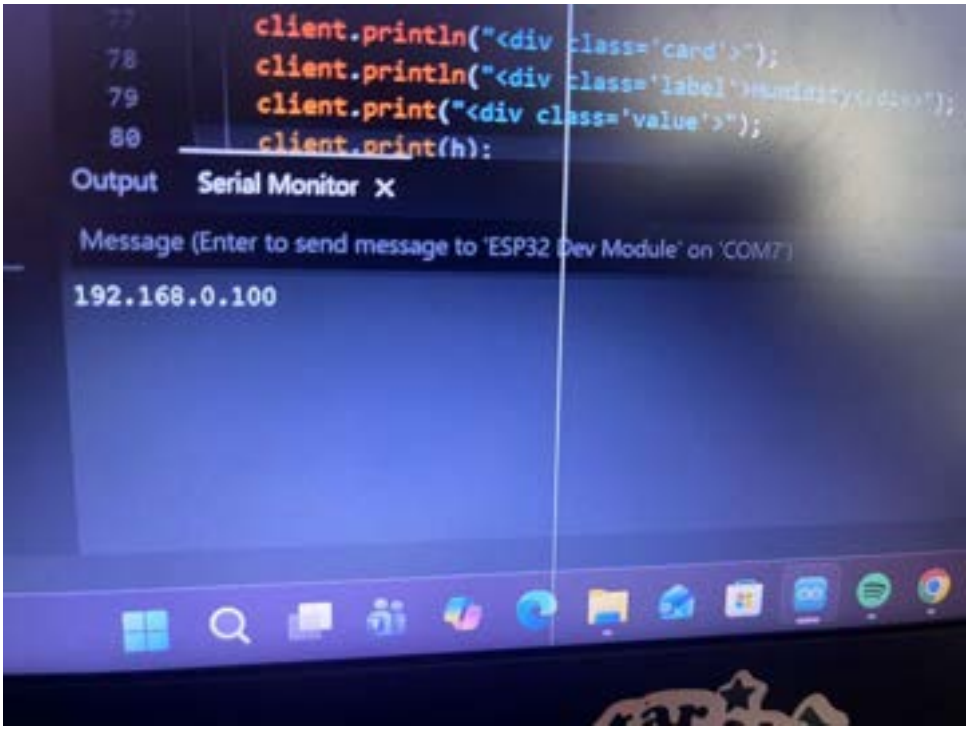
**Step 7:**Open Arduino IDE,and select Board and Ports,after that type the code on Arduino IDE

**Step 8:**Upload the code




**Step 9:**

Now see the ip in serialmonitor




**Step 9:Now open the same ip address in a searchbar and connect the same wifi on that phone**

00:37
192.168.0....
168.0.100
AIR QUALITY MONITOR
Temperature
31.40 C
Humidity
59.00 %
Air Quality
687

# FUTURE SCOPE AND CONCLUSION

## Future Scope :

The **IoT-Based Air Quality Monitoring System** provides a strong base for future improvements and advanced environmental monitoring applications. Several enhancements can be implemented to improve accuracy, usability, and functionality.

**Future improvements could include:**

- **Advanced Sensors:** More specialized sensors like PM2.5 (particulate matter), CO (Carbon Monoxide), or O3 (Ozone) sensors can be added to monitor a wider range of pollutants.
- **Mobile Application Integration:** A dedicated mobile application can be developed to display real-time air quality data, provide safety alerts, and show historical trends for a better user experience.
- **AI and Data Analytics:** Machine learning algorithms can be used to analyze pollution patterns, predicting times of day when air quality might be poorest based on historical data.
- **Real-Time Alerts:** The system can be enhanced to send instant alerts via SMS, email, or push notifications to a smartphone if gas levels exceed hazardous thresholds.
- **Automated Ventilation:** The system could be integrated with smart home devices to automatically turn on an exhaust fan or air purifier when "Unhealthy" air quality is detected.
- **Solar Power Integration:** To make the device truly portable and eco-friendly, a solar panel and battery management system can be added for outdoor monitoring.

## Conclusion

The **IoT-Based Air Quality Monitoring System** successfully achieves its objective of measuring and monitoring environmental parameters in real time. The system detects temperature and humidity using the **DHT11** sensor and harmful gases using the **MQ135** sensor, processing the data through the **ESP32** microcontroller.

This project demonstrates the effective use of IoT technology in environmental safety applications and provides valuable hands-on experience in embedded systems, sensor calibration, and Wi-Fi-based data processing. It is especially useful for students to understand the working of chemical sensors, IoT communication, and the importance of air quality awareness.

It is important to note that this project is developed for educational, testing, and demonstration purposes only. The system is not intended for professional industrial safety or official environmental reporting, as it does not meet laboratory-grade accuracy and certification standards.

Overall, due to its simplicity, low cost, and expandability, this project serves as an excellent learning platform for students and beginners. With further improvements and high-precision components, it can be extended into a more advanced environmental safety system.

# REFERENCES

## References :


 **YouTube Reference –** Project reference, circuit logic, and implementation guidance were taken from educational IoT tutorials.

- *URL: (Insert the specific YouTube link you used, or use your Heart Rate one as a placeholder if same channel)*

 **Arduino IDE –** Software used for writing, compiling, and uploading the program to the ESP32 board.

- URL: https://www.arduino.cc/en/software

 **Sensor Datasheets & Circuit Connections –** Technical information regarding the **MQ135** gas sensor and **DHT11** temperature sensor was referred from online resources and Google search.

 **Mentor Guidance –** Guidance and support were provided by **Sahil Sir** during the design, development, and testing phases of the project.

 **Components Purchased – ESP32**, **MQ135** sensor, **DHT11** sensor, breadboard, and jumper wires were purchased from local electronics stores.

# GLOSSARY

## Glossary :

**ESP32 (NodeMCU):** A powerful, low-cost WiFi and Bluetooth-enabled microcontroller used as the main controller in this project. It reads environmental sensor data, processes air quality values, and manages communication.

**MQ135 Sensor:** A hazardous gas sensor used to detect pollutants such as $CO_2$, $NH_3$, $NO_x$, alcohol, benzene, and smoke. It provides analog signals that the controller converts into air quality levels.

**DHT11 Sensor:** A digital sensor used to measure ambient temperature and humidity. It provides stable digital signals to the ESP32 for climate monitoring.

**Arduino IDE:** An Integrated Development Environment used to write, compile, and upload the C++ programs to the ESP32 microcontroller.

**WiFi:** A wireless networking technology that allows the ESP32 to connect to a local network for data transmission and remote monitoring capabilities.

**PPM (Parts Per Million):** The unit used to measure the concentration of gases in the air. Higher PPM values indicate poorer air quality.

**Analog Pin (GPIO 34):** A specific pin on the ESP32 used to read the varying analog voltage signals from the MQ135 sensor to determine gas concentration.

**Digital Pin (GPIO 4):** A pin on the ESP32 used to receive the digital data stream from the DHT11 sensor.

**IoT (Internet of Things):** The technology that enables your air quality monitor to connect to the network and share environmental data with other devices or cloud services.

**Threshold Value:** A predefined gas concentration level (e.g., 400 PPM) used to trigger an "Unhealthy" status alert when detected by the system.

**Serial Monitor:** A tool within the Arduino IDE used to view real-time text output from the ESP32, essential for debugging and viewing sensor data during the project's testing phase.

**Breadboard & Jumper Wires:** Hardware tools used to create temporary electrical connections between the ESP32 and sensors without the need for soldering.